TECHNICAL OVERVIEW

AI BA Helper

An AI-assisted Business Analyst platform that turns raw requirements into user stories, wireframes, clickable multi-screen prototypes, marketing collateral, and solution architecture.

React 19 + Vite | Express 5 (ESM) | MongoDB + Mongoose | JWT + RBAC | Ollama / Groq / OpenRouter

---

Document: Codebase technical overview & architecture reference

Repository: ai-ba-helper

Generated: June 14, 2026

Audience: Engineers, architects, technical reviewers

Contents

1 Executive Summary

> AI BA Helper is a full-stack web application that compresses the early discovery work of a Business Analyst. A user pastes or uploads raw requirements, and the system uses Large Language Models to produce structured, review-ready artifacts: user stories with acceptance criteria and tests, UI wireframes, interactive multi-screen HTML prototypes, marketing collateral, and solution-architecture documents.

1.1 Problem it solves

Translating stakeholder requirements into well-formed user stories, mockups, and supporting documents is slow, repetitive, and inconsistent. AI BA Helper standardizes the output (a fixed story schema with acceptance criteria, business rules, validations, fields, Definition of Ready / Done) and automates the generation of visual artifacts directly from those stories.

1.2 Core capabilities

Capability	Description
Requirement → Stories	Raw text or large documents are chunked and enhanced by an LLM into a structured set of user stories (acceptance criteria, happy/negative tests, fields, business rules, validations, edge cases, DoR/DoD).
Story → Wireframe	Each story can be turned into a self-contained HTML/CSS/JS wireframe that implements its fields and validation rules.
Stories → Prototype	Multiple stories are assembled into a themed, multi-screen, clickable HTML application prototype via a background job pipeline.
Prototype merge / update	Existing prototypes can be merged into one unified app, or iteratively updated with a natural-language change request.
Marketing collateral	Generates structured marketing content (one-pagers) from stories and exports to PDF / DOCX / PPTX.
Solution architecture	Produces a solution-architecture document from the project's stories.
Administration	Projects, users, and roles with fine-grained module/action permissions and an analytics dashboard.

1.3 Shape of the system

The codebase is a two-package monorepo: a `client/` single-page React application and a `server/` Express REST API backed by MongoDB. The server brokers all LLM calls through a provider-agnostic AI layer that supports a local **Ollama** runtime, **Groq**, and **OpenRouter**, with automatic model fallback and retry logic. The heaviest workload — multi-screen prototype generation — runs asynchronously as a tracked, resumable-by-polling background job.

2 Technology Stack & Dependencies

2.1 Client (`client/package.json`)

Package	Version	Role
`react` / `react-dom`	^19.1.0	UI runtime.
`react-router-dom`	^7.6.3	Client-side routing and nested protected routes.
`axios`	^1.10.0	HTTP client with a shared instance + JWT interceptor.
`lucide-react`	^0.525.0	Icon set.
`vite`	^7.0.0	Dev server and build tooling.
`tailwindcss` (+ `@tailwindcss/postcss`)	^4.1.11	Utility-first styling via PostCSS.
`eslint` + plugins	^9.29.0	Linting (react-hooks, react-refresh).

2.2 Server (`server/package.json`)

Package	Version	Role
`express`	^5.1.0	HTTP framework (Express 5).
`mongoose`	^8.16.1	MongoDB ODM (schemas, models, indexes).
`jsonwebtoken`	^9.0.3	JWT signing/verification for auth.
`bcryptjs`	^3.0.3	Password hashing.
`axios`	^1.10.0	Outbound calls to AI providers.
`openai`	^5.8.2	OpenAI SDK (present; main path uses raw HTTP via axios).
`cors`	^2.8.5	Cross-origin access for the SPA.
`dotenv`	^17.0.1	Environment configuration.
`docx`	^9.7.1	DOCX export of marketing collateral.
`pdfkit`	^0.19.1	PDF export of marketing collateral.
`pptxgenjs`	^4.0.1	PPTX export of marketing collateral.
`nodemon` (dev)	^3.1.14	Auto-restart in development.

Module system
Both packages declare `"type": "module"`. The server uses explicit `.mjs` extensions throughout and native ES module imports.

3 Repository Structure

The repository splits cleanly into `client/` and `server/`. Source files (excluding `node_modules/`) are laid out as follows.

```
ai-ba-helper/
├─ client/                          # React 19 + Vite SPA
│  ├─ index.html                    # Vite entry HTML
│  ├─ vite.config.js  postcss.config.js  tailwind.config.js
│  └─ src/
│     ├─ main.jsx                   # App bootstrap (providers, router)
│     ├─ App.jsx                    # Route table + RBAC guards
│     ├─ api/axiosInstance.js       # Axios + JWT interceptor + poll helper
│     ├─ context/                   # AuthContext, ThemeContext
│     ├─ components/                # ProtectedRoute, Pagination, ChangePasswordModal
│     ├─ pages/                     # One file per screen (Dashboard, Login, ...)
│     │  ├─ components/             # TestCaseEditor, UIWireframeGenerator
│     │  └─ layout/MainLayout.jsx   # Shell: sidebar + outlet
│     └─ utils/pagination.js
│
└─ server/                          # Express 5 REST API (ESM)
   ├─ index.mjs                     # App bootstrap, route mounting, DB connect
   ├─ .env                          # Runtime config (committed — see §14)
   ├─ constants/modules.mjs         # RBAC modules + permission builders
   ├─ middleware/auth.middleware.mjs  # authenticate + authorize
   ├─ routes/*.routes.mjs           # 10 route groups
   ├─ controllers/*.controller.mjs  # Request handlers + AI orchestration
   ├─ models/*.mjs                  # Mongoose schemas (8 collections)
   ├─ services/prototypeJobRunner.mjs # Async prototype pipeline
   ├─ seed/seed.mjs                 # Super Admin role + admin user seeding
   └─ utils/                        # AI parsing, chunking, prototype builders, exports
```

3.1 Naming conventions

- **Routes** are mounted under `/api/<group>` and named `<group>.routes.mjs`.
- **Controllers** mirror routes (`<group>.controller.mjs`) and hold both CRUD and AI-orchestration logic.
- **Models** are singular PascalCase Mongoose models in `<name>.model.mjs` (with the exception of `projects.mjs`).
- **Utilities** are grouped by concern (`aiParser`, `contentChunker`, `prototype*`, `marketingExport`, `openrouter`, `permissions`).

Observed inconsistencies

A few files indicate work-in-progress or duplication: `client/src/pages/UserStoryPage copy.jsx`, and the server's `ai.controller.openai.mjs` and `ai.controller.willbelater.mjs` are not wired into routes. These are flagged in §14.

4 Architecture Overview

AI BA Helper follows a classic decoupled SPA + REST API architecture, with a third dependency tier for AI providers. The React client talks to the Express API exclusively over JSON; the API persists to MongoDB and proxies all model calls to a configurable AI provider.

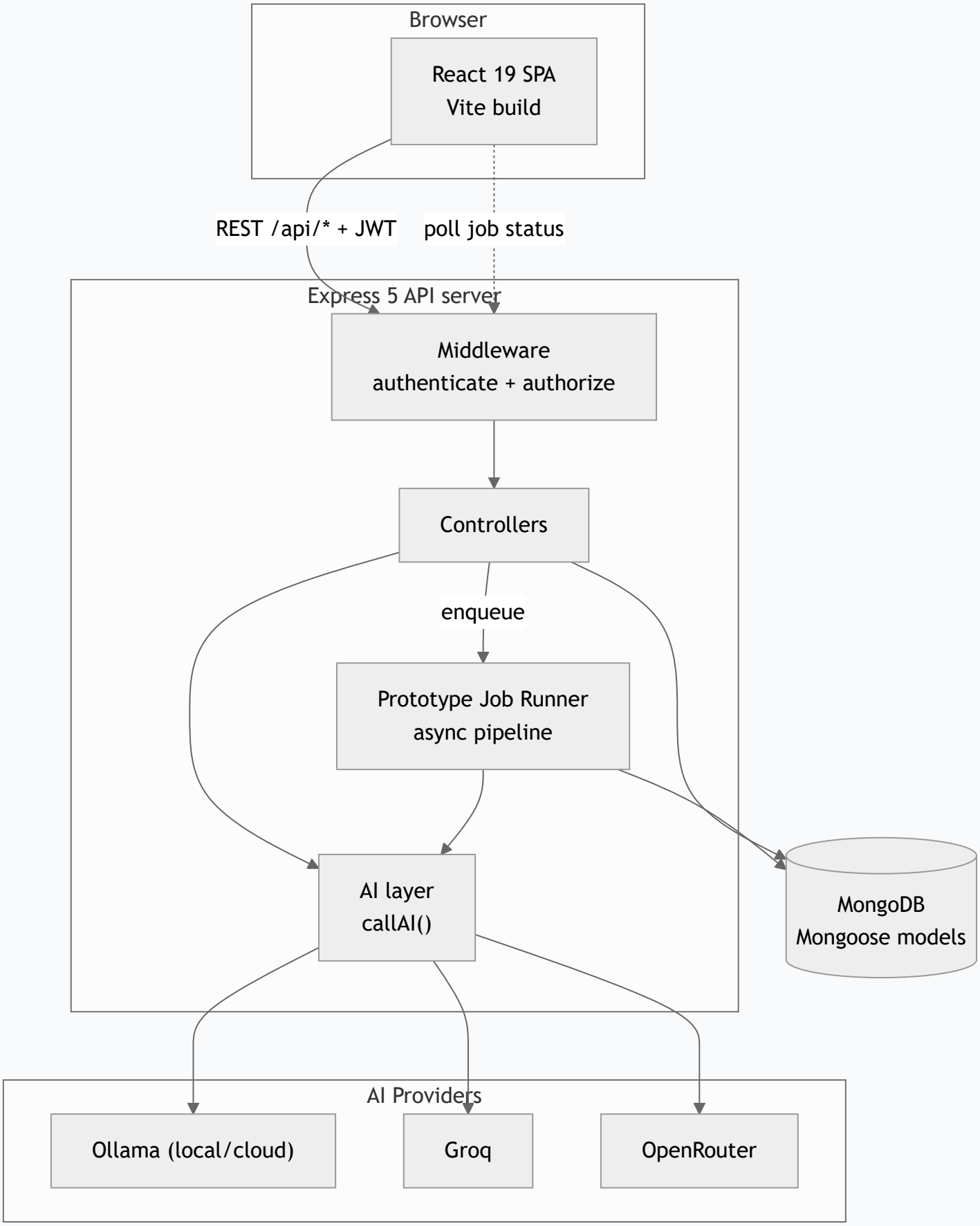

Figure 4.1 — High-level runtime topology. The SPA polls the API for long-running prototype jobs.

4.1 Layering

Layer	Responsibility
Presentation	React pages + components; route-level RBAC; axios instance with token injection and 401 handling.
Transport / Routing	Express routers under `/api`; per-route `authenticate` then `authorize(module, action)`.
Application / Controllers	Validation of input, orchestration of AI calls, persistence, export generation.
AI orchestration	Provider selection, model fallback, prompt assembly, JSON parsing/repair, retries with backoff.
Async processing	Prototype generate/merge/update jobs executed off the request thread, progress persisted to Mongo.
Persistence	Mongoose models; ownership stamped via `createdBy`; TTL index expires jobs after 24h.

Design intent

Controllers are deliberately "thick" — they combine HTTP handling, AI prompt construction, and persistence. The AI complexity is factored out into `controllers/ai.controller.mjs` and the `utils/prototype*` family so feature controllers can reuse `callAI()` and the builder helpers.

5 Client Architecture

5.1 Bootstrap & providers

`main.jsx` mounts the app inside the router and the `AuthProvider` / `ThemeProvider` contexts. `App.jsx` declares the entire route table and wraps protected areas in nested `<ProtectedRoute>` elements.

5.2 Routing & route-level RBAC

Every authenticated route is double-guarded: an outer `<ProtectedRoute>` ensures a session exists, and an inner one checks a specific `module` / `action` permission. Unauthorized users are redirected to their first permitted route.

```
<Route element={<ProtectedRoute module="stories" action="read" />}>
  <Route path="/add-user-story" element={<UserStoryPage />} />
  <Route path="/user-stories"   element={<UserStories />} />
</Route>
```

Route	Page	Required permission
`/login`	Login	Public
`/dashboard`	Dashboard	`dashboard:read`
`/add-project`	AddProject	`projects:read`
`/add-user-story` , `/user-stories`	UserStoryPage, UserStories	`stories:read`
`/create-application`	CreateApplication	`applications:read`
`/marketing-collateral`	MarketingCollateral	`marketing:read`
`/solution-architecture`	SolutionArchitecture	`solution:read`
`/roles`	Roles	`roles:read`
`/users`	Users	`users:read`

5.3 Authentication context

`AuthContext` loads the current user from `GET /auth/me` on mount (using a token in `localStorage`), and exposes `login`, `logout`, `hasPermission`, and `getDataAccess`. Permission checks read the role's embedded `permissions` array, so the UI hides/shows features in lockstep with the server's RBAC model.

5.4 API layer

The shared `axiosInstance` centralizes cross-cutting HTTP behavior:

- **Base URL** `http://localhost:5000/api` with a **10-minute timeout** — intentionally long because local Ollama + large-document processing can be slow.
- **Request interceptor** injects `Authorization: Bearer <token>` from `localStorage`.
- **Response interceptor** on a `401` clears the token and redirects to `/login` (except for the login call itself).
- `pollWithTimeout` helper issues short-timeout GETs, used to poll prototype-job progress.

5.5 Pages

Page	Responsibility
Login	Email/password sign-in; stores JWT.
Dashboard	Aggregate stats via `/dashboard/stats`.
AddProject	Create/select the project that scopes all artifacts.
UserStoryPage	Paste/upload requirements, run AI enhancement, edit the structured story (uses `TestCaseEditor`, `UIWireframeGenerator`).
UserStories	Browse, paginate, and manage saved stories.
CreateApplication	Select stories, write a brief, launch and watch a prototype job, preview/merge/update prototypes.
MarketingCollateral	Generate and export collateral (PDF/DOCX/PPTX).
SolutionArchitecture	Generate a solution-architecture document from stories.
Users / Roles	Admin CRUD for users and permission-bearing roles.

6 Server Architecture

6.1 Bootstrap (`server/index.mjs`)

The entry point configures Express, mounts the ten route groups, connects to MongoDB, and — only after a successful connection — recovers stale jobs, seeds the database, and starts listening.

```
app.use(express.json({ limit: "15mb" }));   // large requirement payloads
app.use(cors());                             // open CORS

app.use("/api/auth", authRoutes);
app.use("/api/roles", roleRoutes);
app.use("/api/users", userRoutes);
app.use("/api/stories", storyRoutes);
app.use("/api/ai", aiRoutes);
app.use("/api/projects", projectRoutes);
app.use("/api/dashboard", dashboardRoutes);
app.use("/api/applications", prototypeRoutes);
app.use("/api/marketing", marketingRoutes);
app.use("/api/solutions", solutionRoutes);

mongoose.connect(process.env.MONGO_URI).then(async () => {
  await markStaleJobsFailed();   // recover interrupted jobs
  await seedDatabase();          // ensure Super Admin + admin user
  app.listen(process.env.PORT || 5000);
});
```

Request size

The JSON body limit is raised to `15mb` to accommodate pasted documents and prototype HTML payloads.

6.2 Request lifecycle

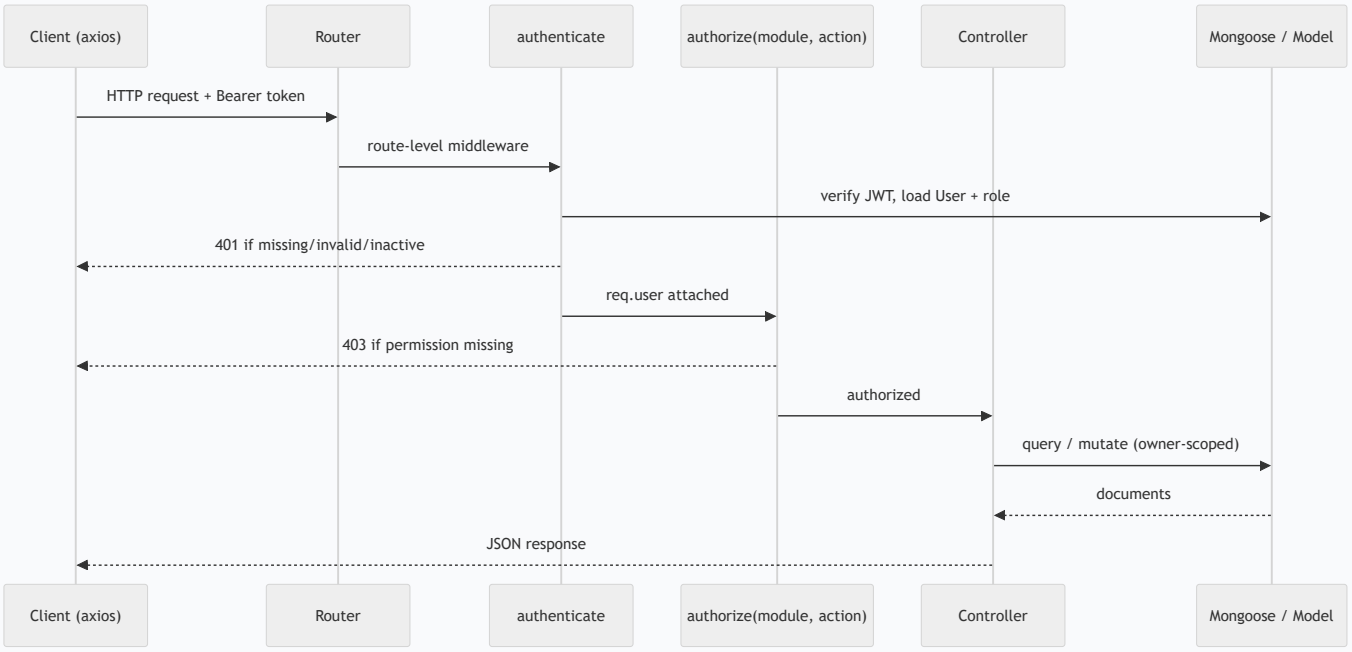

Figure 6.1 — Standard authenticated request path. Most routers call `router.use(authenticate)` once, then apply `authorize()` per endpoint.

6.3 API surface

All endpoints below require a valid JWT; the listed module/action is enforced by `authorize()`.

AUTH — `/api/auth`

`POST` `/login`	Public. Returns JWT + user.
`GET` `/me`	Current user (auth only).
`POST` `/change-password`	Change own password (auth only).

STORIES — `/api/stories` & AI — `/api/ai`

`POST` `/stories`	`stories:create`
`GET` `/stories`, `/stories/:id`, `/stories/search/by-project`	`stories:read`
`PUT` `/stories/:id`	`stories:update`
`DELETE` `/stories/:id`	`stories:delete`
`POST` `/stories/enhance`	`stories:update`
`POST` `/ai/enhance`	`ai:create` — requirements → stories.
`POST` `/ai/generate-ui`	`ai:create` — story → wireframe.

APPLICATIONS / PROTOTYPES — `/api/applications`

`GET` `/project/:projectId/stories`	Preview stories for selection (`applications:read`).
`POST` `/refine-prompt`, `/prompt/refine`	AI-refine the prototype brief (`applications:create`).
`POST` `/generate`, `/merge`, `/update`	Enqueue generate / merge / update jobs.
`GET` `/jobs/:jobId`	Poll job progress.
`DELETE` `/jobs/:jobId`	Cancel a running job.
`GET` `/`, `/:id` `POST` `/` `PUT` `/:id` `DELETE` `/:id`	CRUD on saved prototypes.

MARKETING — `/api/marketing` & SOLUTIONS — `/api/solutions`

`POST` `/marketing/generate`, `/solutions/generate`	Generate content from stories.
`POST` `/marketing/export`	Export collateral to PDF/DOCX/PPTX.
`POST` `/{marketing,solutions}/prompt/refine`	Refine the generation brief.
CRUD `/`, `/:id`	Standard create/read/update/delete.

ADMIN — `/api/projects`, `/api/users`, `/api/roles`, `/api/dashboard`

Projects	CRUD + `GET /search`.
Users	CRUD (`users:*`).
Roles	CRUD + `GET /modules` (catalog of permissionable modules).
Dashboard	`GET /stats` (`dashboard:read`).

7 Authentication & Authorization

7.1 Authentication

Login compares a bcrypt-hashed password and issues a JWT signed with `JWT_SECRET` that expires in **7 days**. The `authenticate` middleware extracts the `Bearer` token, verifies it, loads the user with its `role` populated, and rejects missing/invalid tokens or inactive users.

```
export const signToken = (userId) =>
  jwt.sign({ userId }, JWT_SECRET, { expiresIn: "7d" });

export const authenticate = async (req, res, next) => {
  const header = req.headers.authorization;          // "Bearer <token>"
  const decoded = jwt.verify(token, JWT_SECRET);
  const user = await User.findById(decoded.userId).populate("role");
  if (!user || !user.isActive) return res.status(401)...;
  req.user = user; next();
};
```

Passwords are hashed in a Mongoose `pre("save")` hook and never selected by default (`select: false`):

```
userSchema.pre("save", async function () {
  if (!this.isModified("password")) return next();
  this.password = await bcrypt.hash(this.password, 10);
});
```

7.2 Authorization model (RBAC)

Authorization is module + action based. The catalog of modules lives in `constants/modules.mjs`; each **Role** embeds a `permissions` array with `create/read/update/delete` booleans and a `dataAccess` scope (`own` vs `all`) per module.

Modules	Actions	Data access
dashboard, projects, stories, applications, marketing, solution, ai, users, roles	create, read, update, delete	`own` (only `createdBy` = self) or `all`

```
// authorize() gate, applied per route
export const authorize = (module, action) => (req, res, next) => {
  if (!hasPermission(req.user, module, action))
    return res.status(403).json({ error: "..." });
  next();
};

// data scoping inside controllers
export const buildOwnerFilter = (user, module) =>
  hasDataAccessAll(user, module) ? {} : { createdBy: user._id };
```

Defense in depth

The same permission shape is enforced twice: client-side (`AuthContext.hasPermission` hides UI) and server-side (`authorize` blocks the request). Record-level access additionally narrows queries via `buildOwnerFilter` / `canAccessResource`.

7.3 Seeding

On every boot, `seedDatabase()` ensures a **Super Admin** role exists with full permissions and an admin user (`SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD`, defaulting to `admin@bahelper.com`). Existing Super Admin permissions are re-synced to the full set on each start.

8 Data Model

Eight Mongoose collections. Almost every domain document is scoped to a `Project` and stamped with `createdBy` for ownership-based access control.

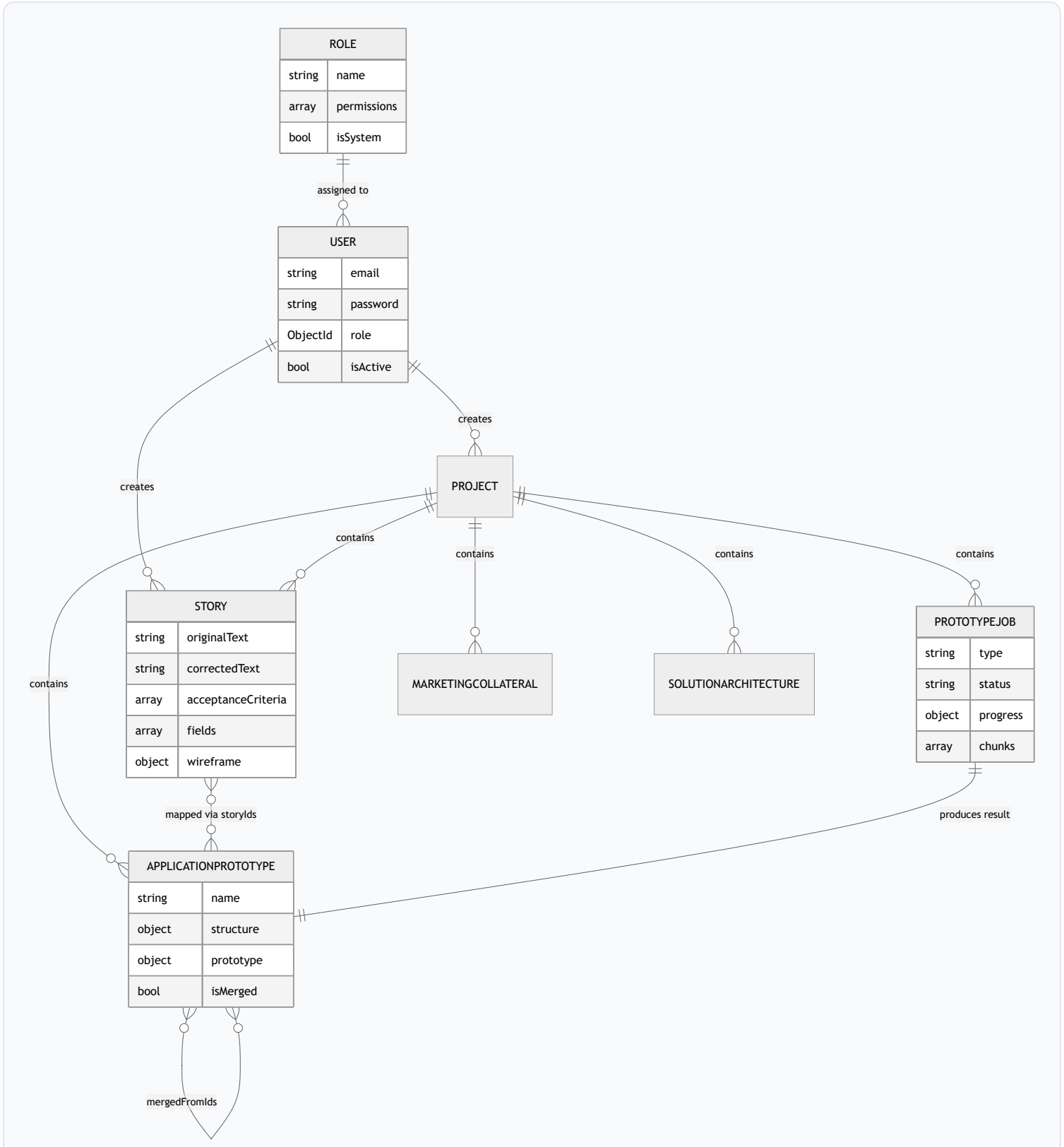

Figure 8.1 — Entity relationships. Story-to-prototype mapping is a soft reference via `storyIds` / `mappedStoryIds`.

8.1 Collection reference

Collection	Key fields	Notes
User	`firstName, lastName, email (unique), password (select:false), role → Role, isActive, createdBy`	bcrypt hash on save; `comparePassword` method.
Role	`name (unique), description, permissions[], isSystem`	Permission sub-schema: module enum + CRUD booleans + `dataAccess`.
Project	`name, description, createdBy`	Top-level scope for all artifacts.
Story	`projectId, originalText, correctedText, feature, acceptanceCriteria[], happyTests[], negativeTests[], fields[], businessRules[], validations[], edgeCases[], constraints[], dependencies[], businessImpact, definitionOfReady[], definitionOfDone[], status, wireframe{}`	Rich BA schema; `status` = draft / reviewed / final; embedded wireframe HTML/CSS/JS.
ApplicationPrototype	`projectId, name, storyIds[], structure{appName, summary, navigation[], screens[], theme}, prototype{html, css, js, fullDocument}, usedScaffold, isMerged, mergedFromIds[]`	Final saved multi-screen app + its structure/theme.
PrototypeJob	`type, status, projectId, storyIds[]/prototypeIds[]/sourcePrototypeId, prompts, progress{phase, percent, message}, chunks[], screenChunks[], partialPrototype{}, result, cancelled`	Tracks generate/merge/update. **TTL index expires after 86,400s (24h)**; indexed on `(projectId, createdBy, status)`.
MarketingCollateral	`projectId, name, collateralType, format(pdf/docx/pptx), storyIds[], content{title, sections[], callToAction, theme}, hasContent`	Structured content suitable for export.
SolutionArchitecture	`projectId, name, storyIds[], prompt, content (Mixed), hasContent`	Flexible content blob from AI.

9 AI Integration Layer

> `controllers/ai.controller.mjs` is the heart of the system: a single resilient `callAI(systemPrompt, userPrompt, useJsonMode, options)` function abstracts three providers, multiple models, JSON-mode quirks, reasoning toggles, retries, and backoff behind one interface.

9.1 Provider abstraction

The active provider is chosen from env (`AI_PROVIDER` , default `ollama`); the prototype pipeline can use a separate provider (`PROTOTYPE_AI_PROVIDER`). Each provider has its own endpoint, headers, and model defaults.

Provider	Endpoint	Auth / notes
Ollama	`OLLAMA_BASE_URL` (default `localhost:11434/v1`)	No key; supports local and `:cloud` models; long timeouts.
Groq	`api.groq.com/openai/v1`	`GROQ_API_KEY` ; low concurrency to respect free-tier rate limits.
OpenRouter	`openrouter.ai/api/v1`	`OPENROUTER_API_KEY` ; rotates through a list of free models.

9.2 Resilience strategy

A single call can traverse multiple models and multiple per-model attempts:

- **Model fallback** — ordered lists (e.g. configured model → `FREE_MODELS` / `UI_MODELS`); rate-limited models are skipped to the next.
- **Attempt matrix** — `buildAttempts()` tries combinations of JSON mode on/off and reasoning-exclude on/off, tailored per model family (Nemotron/Gemma/cloud vs Hermes/free).
- **Transparent retry** — `postWithRetry()` retries `429/500/502/503` with exponential backoff, honoring the provider's `Retry-After` header.
- **Content extraction** — `extractMessageContent()` picks the first non-empty of `message.content` , `reasoning` , or `output_text` .
- **Actionable errors** — exhausted/unreachable providers yield human-readable guidance (e.g. "Ollama is not reachable", "OpenRouter free daily quota is exhausted").

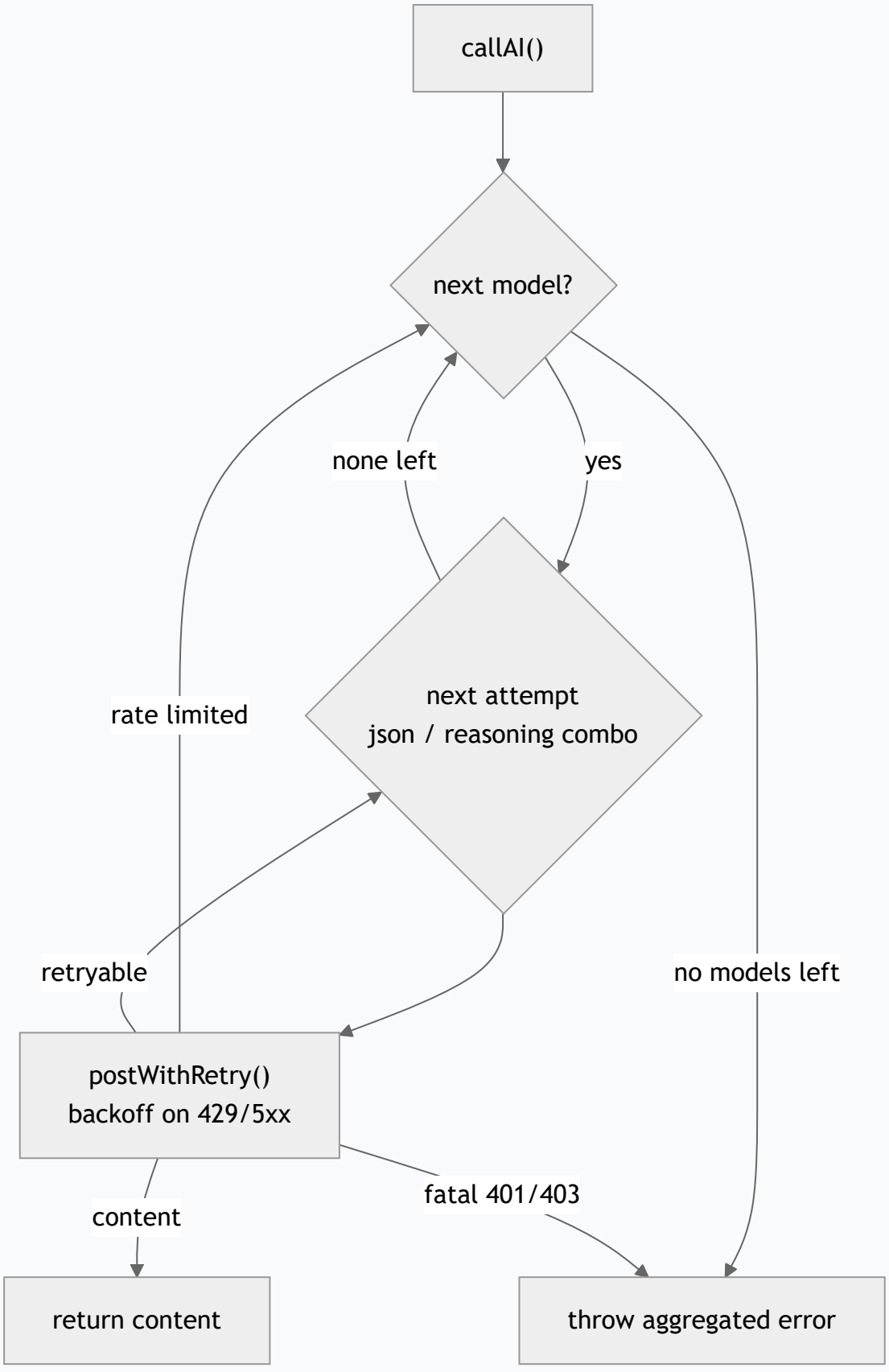

Figure 9.1 — `callAI` control flow across models and attempts.

9.3 Story enhancement

`enhanceStory()` chunks large input (`contentChunker`), and for each chunk asks the model for strictly-typed JSON user stories with a fixed schema (acceptance criteria, happy/negative tests, fields, business rules, validations, edge cases, DoR/DoD). Results are parsed with repair logic (`parseAIJson`),

validated (`validateStories`), de-duplicated, and merged. Invalid JSON triggers a corrective re-prompt.

9.4 Wireframe generation

`generateUI()` turns a single story into a self-contained HTML/CSS/JS wireframe that renders a real input for every declared field, wires the validation rules, and respects business rules/constraints — or returns `{ applicable: false }` for backend-only stories. Output is sanitized (`sanitizePrototypeCode`) and wrapped into a full HTML document.

Option presets

Tuned option builders — `getUIAIOptions` , `getEnhanceAIOptions` , `getStructureAIOptions` , `getScreenAIOptions` — set per-task model lists, token budgets, and timeouts so each step uses an appropriate model and limit.

10 Prototype Generation Engine

> `services/prototypeJobRunner.mjs` is the most sophisticated subsystem. Because a multi-screen prototype can take minutes, generation runs as an asynchronous, persisted, cancellable job built from discrete *chunks*; the client polls `GET /applications/jobs/:jobId` for progress and a live partial preview.

10.1 Job types

Type	Input	Output
generate	Selected stories + optional brief	A new themed multi-screen prototype.
merge	2+ existing prototypes	One unified app reusing source UI.
update	A saved prototype + change request	Edited prototype, screen by screen.

10.2 Generate pipeline

1. **Scaffold** — infer the application domain and an AI-designed `theme` (colors/fonts), then deterministically plan screens and chunks from the stories (`planPrototypeChunks`) and build a preview scaffold.
2. **Structure refine** — for larger/brief-driven jobs, an LLM refines the screen map (grouping stories into 3–12 coherent screens). Small story sets skip this step (`STRUCTURE_SKIP_THRESHOLD`).
3. **Per-screen enrichment** — each screen is generated concurrently (a bounded `runPool`, default concurrency 3) into scoped HTML/CSS/JS; weak/empty output triggers a regenerate, then a scaffold fallback.
4. **Assemble** — screens are injected into the shell, CSS merged, and a final `fullDocument` is composed and stored on the job's `result`.

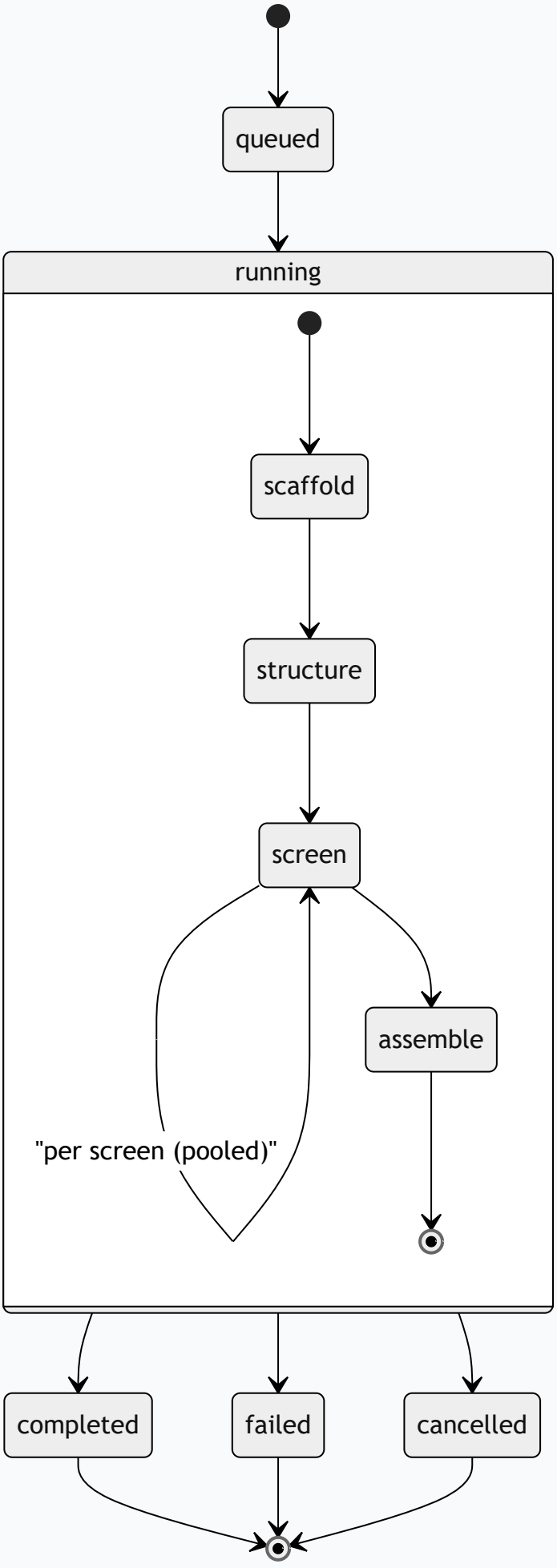
queued
running
scaffold
structure
screen
assemble
"per screen (pooled)"
completed
failed
cancelled

Figure 10.1 — Prototype job state machine. Phases map to chunks persisted on the job document.

10.3 Progress, cancellation & recovery

- **Progress** — each chunk transition updates `progress.percent` / `message` and a `partialPrototype` the UI can render live.
- **Cancellation** — `isJobCancelled()` is checked between phases/chunks; `DELETE /jobs/:jobId` sets the flag.
- **Concurrency guard** — an in-memory `activeJobs` Set prevents a job from running twice in one process.
- **Stale recovery** — on boot, `markStaleJobsFailed()` fails any `queued/running` job older than one hour (e.g. after a server restart).
- **TTL** — job documents auto-expire after 24 hours via a Mongo TTL index.

Single-process assumption

Both the `activeJobs` guard and the `setImmediate`-based execution assume one server instance. Horizontal scaling would require an external queue — see §14.

11 Feature Module Walkthroughs

USER STORIES

UI: UserStoryPage / UserStories
API: `/ai/enhance`, `/stories` CRUD, `/ai/generate-ui`
Model: Story

Paste requirements → AI produces structured stories → edit & save → optionally generate a per-story wireframe stored on `story.wireframe`.

APPLICATIONS / PROTOTYPES

UI: CreateApplication
API: `/applications/generate|merge|update`, `/jobs/:id`
Model: PrototypeJob, ApplicationPrototype

Select stories → enqueue job → poll progress + live preview → save, merge, or iterate.

MARKETING COLLATERAL

UI: MarketingCollateral
API: `/marketing/generate`, `/marketing/export`
Model: MarketingCollateral
Util: marketingExport (docx/pdfkit/pptxgenjs)

Generate structured collateral from stories → persist → export to PDF / DOCX / PPTX.

SOLUTION ARCHITECTURE

UI: SolutionArchitecture
API: `/solutions/generate` + CRUD
Model: SolutionArchitecture

Produce a solution-architecture document from a project's stories; store flexible content.

PROJECTS

UI: AddProject
API: `/projects` CRUD + `/search`
Model: Project

Projects are the scoping unit; every story, prototype, and document references a `projectId`.

USERS & ROLES

UI: Users, Roles
API: `/users`, `/roles` (+ `/roles/modules`)
Model: User, Role

Admins define roles with per-module CRUD + data-access scope, then assign them to users.

DASHBOARD

UI: Dashboard **API:** `GET /dashboard/stats` **Permission:** `dashboard:read`

Aggregates counts across projects, stories, prototypes, and other artifacts for an at-a-glance overview.

12 End-to-End Data Flows

12.1 Requirements → User Stories

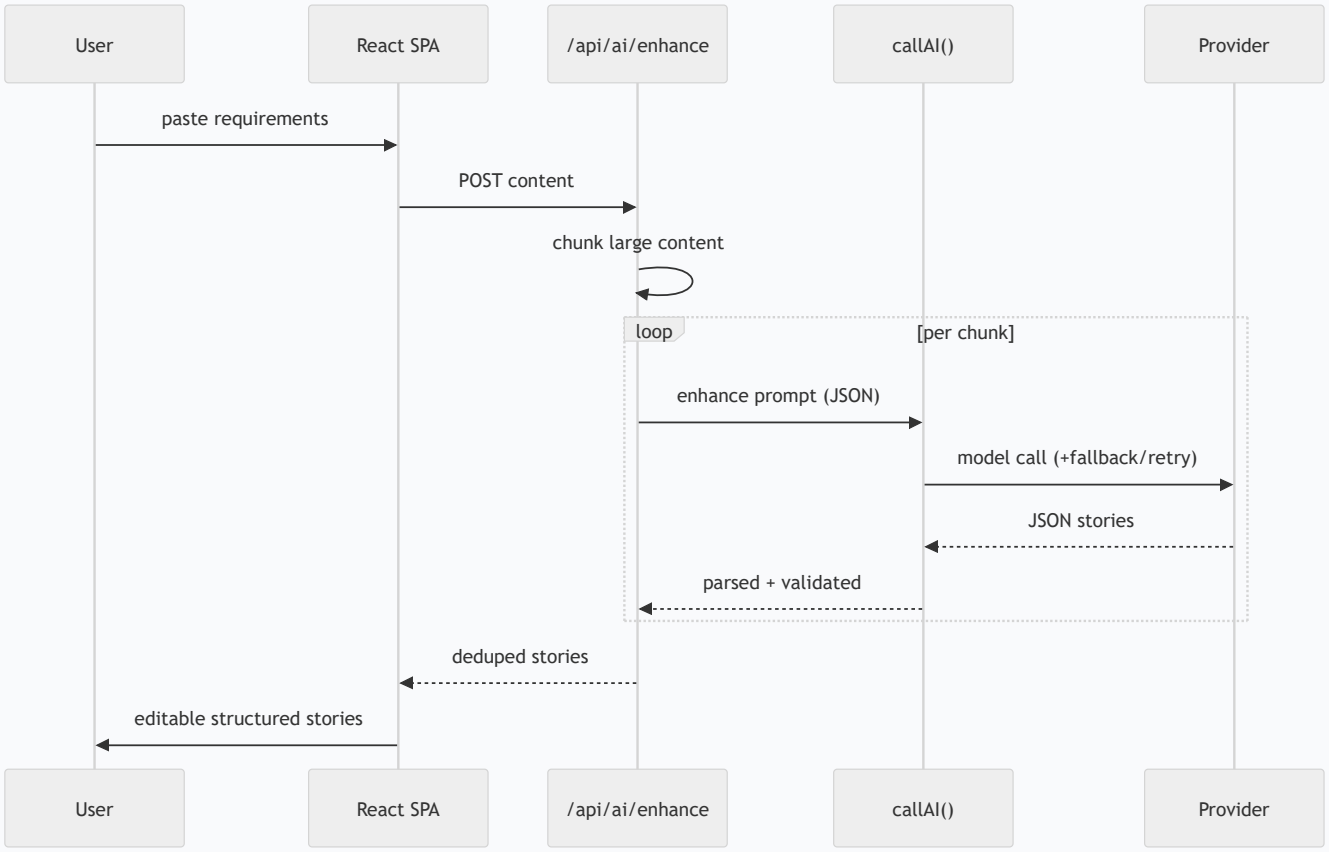

Figure 12.1 — Story generation. Stories are saved separately via `POST /stories` once reviewed.

12.2 Stories → Multi-screen Prototype

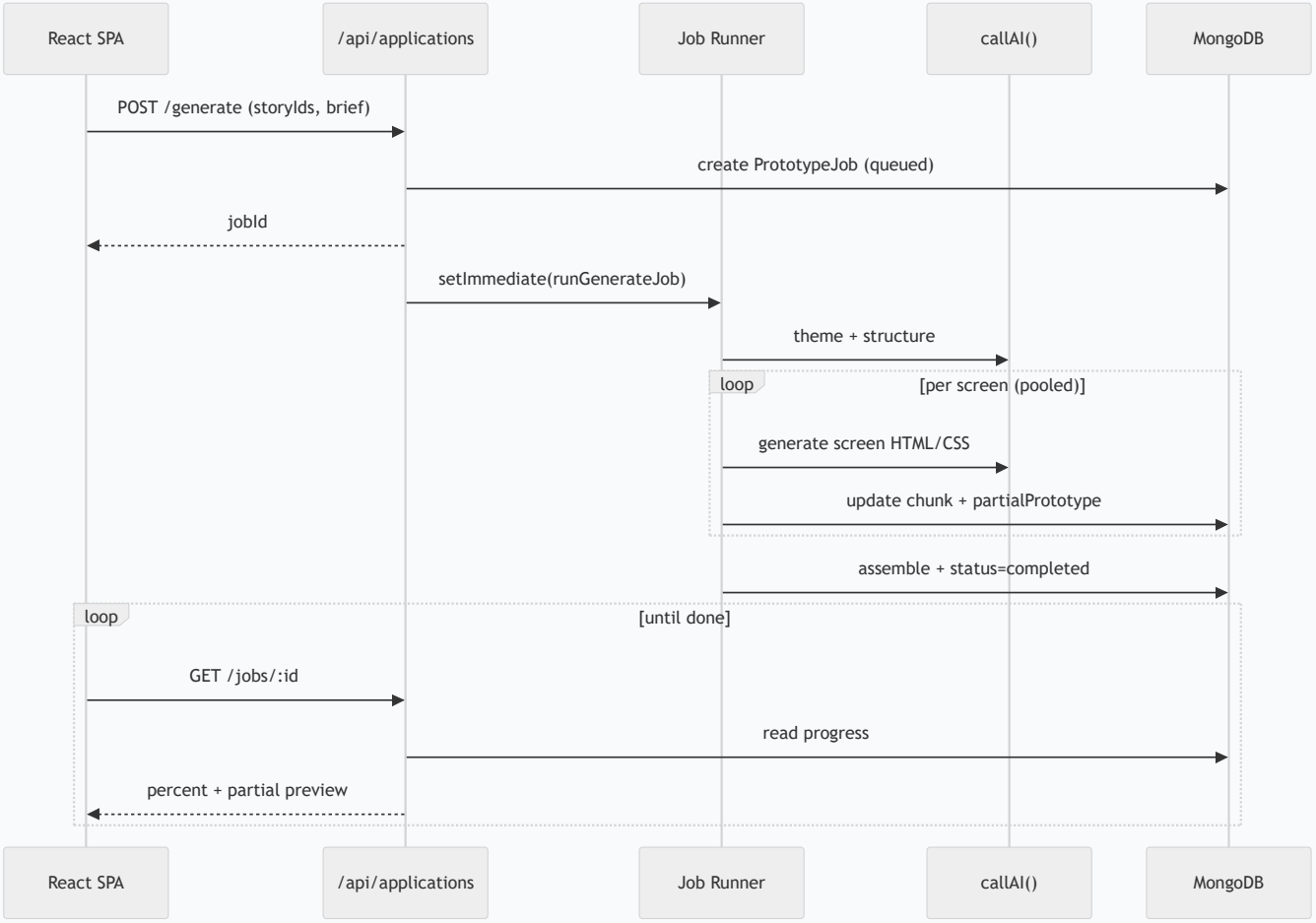

Figure 12.2 — Asynchronous prototype generation with client polling.

13 Configuration & Operations

13.1 Environment variables (`server/.env`)

Variable	Purpose
`MONGO_URI`	MongoDB connection string.
`PORT`	API port (default 5000).
`JWT_SECRET`	JWT signing key (falls back to a hardcoded dev value — see §14).
`AI_PROVIDER` / `PROTOTYPE_AI_PROVIDER`	Active provider(s): `ollama` | `groq` | `openrouter`.
`OLLAMA_BASE_URL`, `OLLAMA_MODEL`, `OLLAMA_UI_MODEL`, `OLLAMA_STRUCTURE_MODEL`, `OLLAMA_PROTOTYPE_MODEL`, `OLLAMA_SCREEN_MODEL`	Ollama endpoint + per-task model selection.
`GROQ_API_KEY`, `GROQ_MODEL`	Groq credentials/model.
`OPENROUTER_API_KEY`, `OPENROUTER_MODEL`, `OPENROUTER_UI_MODEL`	OpenRouter credentials/models.
`*_TIMEOUT_MS`, `PROTOTYPE_SCREEN_CONCURRENCY`, `PROTOTYPE_SCREEN_MAX_TOKENS`	Timeouts, screen concurrency, token budgets.
`SEED_ADMIN_EMAIL`, `SEED_ADMIN_PASSWORD`	Seed admin credentials.

13.2 Running locally

```
# Server
cd server && npm install && npm run dev   # nodemon index.mjs (port 5000)

# Client
cd client && npm install && npm run dev   # vite dev server
```

On first server start, MongoDB is connected, stale jobs are failed, the Super Admin role + admin user are seeded, and the API begins listening. The client expects the API at `http://localhost:5000/api`.

13.3 Operational characteristics

- **Long timeouts** client (10 min) and server are tuned for slow local inference.
- **Self-healing jobs** stale jobs are recovered on boot; jobs TTL-expire after 24h.
- **Graceful AI degradation** model/provider failures fall back rather than hard-failing the request where possible.

14 Areas of Improvement

The following observations are drawn directly from the code. They are grouped by theme and ordered roughly by priority.

14.1 Security & secrets HIGH

- **Committed secrets** — `server/.env` is tracked in git. Secrets (DB URI, API keys) should be removed from history, rotated, and ignored via `.gitignore`.
- **Committed** `node_modules/` — the server's dependencies are checked in, bloating the repo and risking drift. Ignore them and rely on lockfiles.
- **Hardcoded JWT fallback** — `JWT_SECRET` defaults to `"ba-helper-dev-secret-change-in-production"`. The server should refuse to boot without an explicit secret in production.
- **Open CORS** — `cors()` allows any origin. Restrict to known front-end origins.
- **No rate limiting / input validation** — add per-route rate limiting and a schema validator (e.g. `zod` / `express-validator`) for request bodies.

14.2 Code health MEDIUM

- **Dead/duplicate files** — `UserStoryPage copy.jsx`, `ai.controller.openai.mjs`, `ai.controller.willbelater.mjs`, and `server/oldURI.txt` should be removed or folded into the active code.
- **Thick controllers** — feature controllers mix HTTP, prompt building, and persistence. Extracting a service layer would improve testability.
- **No shared contracts** — client and server duplicate the permission/story shapes. Shared types (or an OpenAPI spec) would prevent drift.
- **Configuration sprawl** — many `*_MODEL` / `*_TIMEOUT_MS` knobs are read ad hoc; centralizing config parsing/validation at boot would catch misconfiguration early.

14.3 Reliability & scale MEDIUM

- **In-process jobs** — the `activeJobs` Set and `setImmediate` execution tie jobs to a single instance. A real queue (e.g. BullMQ/Redis) would enable horizontal scaling and durable retries.
- **Polling vs push** — job progress is polled. Server-Sent Events or WebSockets would reduce latency and request volume.
- **No structured logging/metrics** — logging is `console.*`. Structured logs + request tracing would aid production debugging.

14.4 Quality & delivery MEDIUM

- **No automated tests** — the server `test` script is a stub. Add unit tests around `callAI`, parsers, permission logic, and the job state machine.
- **No CI pipeline** — add lint + test + build gates on pull requests.
- **No global error handler** — Express error handling is per-controller; a centralized error middleware would standardize responses.

14.5 Suggested roadmap

Phase	Focus
Now	Purge secrets & `node_modules` from git; enforce required `JWT_SECRET`; lock down CORS; add input validation + rate limiting.
Next	Remove dead code; centralize config; add a global error handler and structured logging; introduce a test suite + CI.
Later	Move prototype jobs to a durable queue; replace polling with SSE/WebSockets; publish shared client/server types or an API spec.

Generated from the `ai-ba-helper` repository. This document reflects the source as read at generation time; re-run `npm run build` in `docs/technical-overview/` to regenerate after code changes.